

도커는 왜 필요한가?

[도커를 사용하면 좋은 점-예시-01]

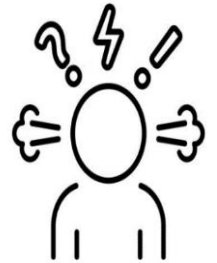


돌리는 서버 100대를 관리한다. 서비스를 하기 위해서 OS를 일일이 설치하고 그 위에 Apache와 Node.js를 설치하고 소스 코드를 다운로드하고 구동시킨다.

→같은 일을 100번 해야 한다.

이번에 Apache를 NginX로 교체하기로 했다. 소스 코드는 Git 등을 통해서 원격으로 쉽게 다운로드가 가능하지만, Apache 나 Nginx 같은 환경에 대한 교체는 쉽지 않다. 일일이 100대의 머신에서 Apache를 제거하고 Nginx를 설치해야 한다. 만약 OS를 CentOS에서 Ubuntu로 교체한다고 한다면 일일이 모두 포맷하고 모든 과정을 반복해야 한다.

Docker를 사용할 경우, CentOS + Apache + Node.js + Source 형식의 이미지를 하나만 준비하고 원격으로 한 번에 모두 배포 및 구동한다. Apache가 Nginx로 교체된다면 CentOS + Nginx + Node.js + Source 의 형태로 이미지만 다시 제작하여 한 번에 배포한 후 컨테이너만 기동시키면 된다. OS의 경우도 마찬가지이다.



극단적으로 예를 들긴 한 것이지만 그 만큼 Docker를 활용하여 단순히 비즈니스 어플리케이션을 넘어서 환경 자체를 배포하기 용이하다.

[도커를 사용하면 좋은 점-예시-02]

만약, 하나의 PC에서 php, 파이썬을 실행 하고 있을 때 둘 다 Mysql을 사용하는데, 서로 다른 버전이 필요하다면 어떻게 해야 하나?

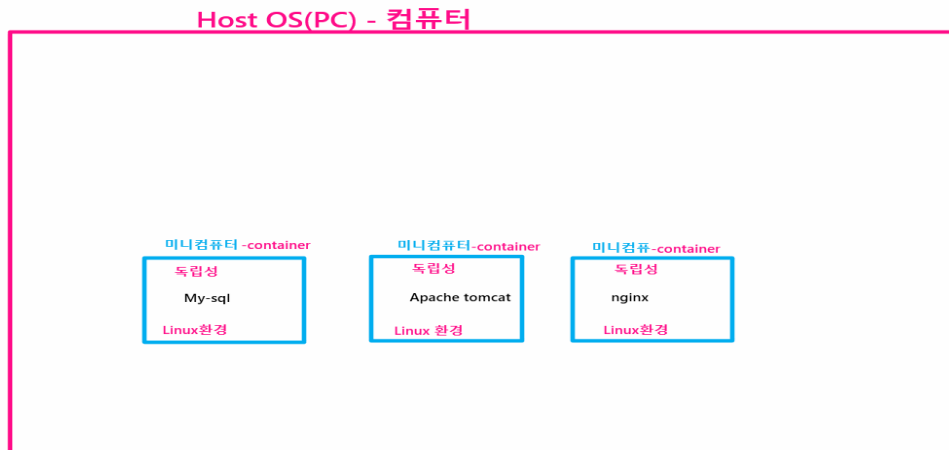
특정 프로그램을 업데이트 하면 다른 프로그램에도 영향을 미칠 수 있다.

버전이 다른 MySQL프로그램을 서로 다른 격리된 환경에서사용 할 수 있다면 서로에게 영향을 주지 않고 다양한 버전을 사용 하고 test 할 수 있을 것이다. 이 것을 가능하게 도와주는 것이 바로 컨테이너기반의 기술 도커가 필요한 이유이다..

Docker를 시작하기 전에 Docker 간단히 정리해보면

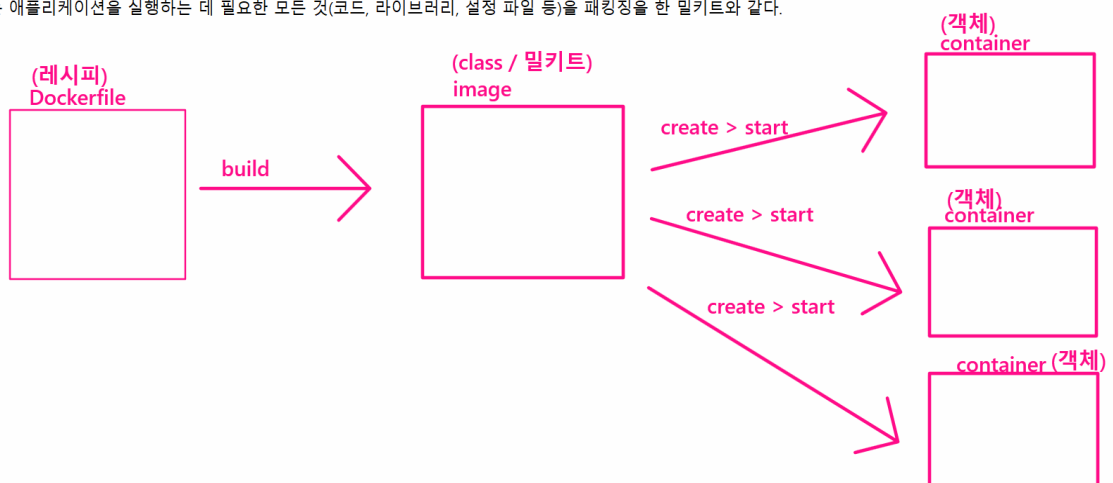
1) 격리된 환경 제공

Docker는 애플리케이션을 실행하는 데 필요한 모든 것(코드, 라이브러리, 설정 파일 등)을 패키징을 한 밀키트와 같다.



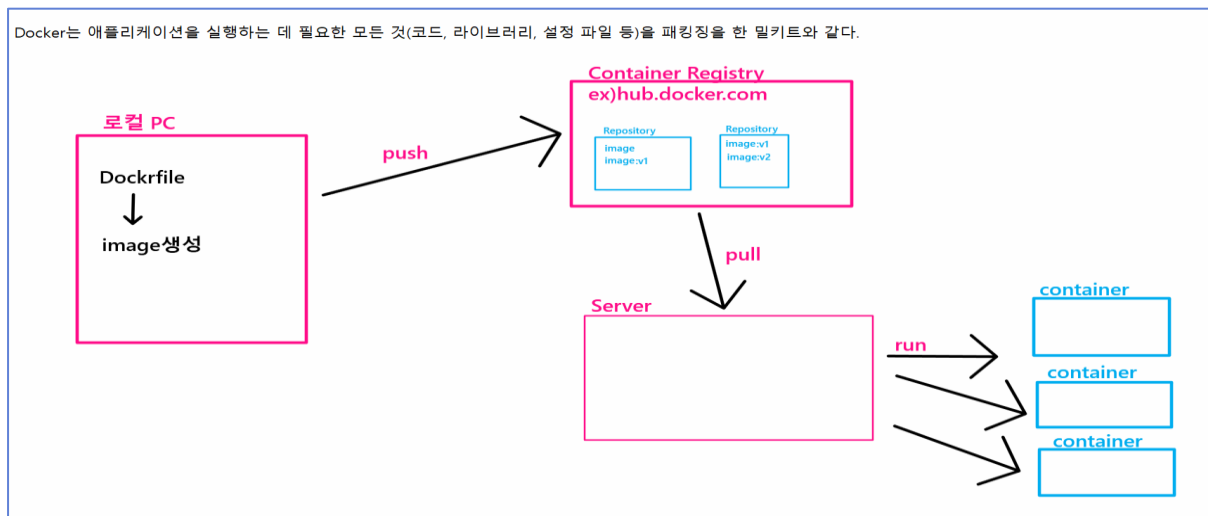
2) Dockerfile을 이용하여 나만의 패키징된 이미지를 만들어 동일한 컨테이너 여러 개 생성

Docker는 애플리케이션을 실행하는 데 필요한 모든 것(코드, 라이브러리, 설정 파일 등)을 패키징을 한 밀키트와 같다.



3) Docker hub 원격 저장소에 image push하여 image 공유

Docker는 애플리케이션을 실행하는 데 필요한 모든 것(코드, 라이브러리, 설정 파일 등)을 패키징을 한 밀키트와 같다.



Docker란?

도커는

- 2013년 3월에 처음으로 세상에 알려졌으며 컨테이너 기반의 오픈소스 가상화 플랫폼이다.

Docker는 애플리케이션을 실행하는 데 필요한 모든 것(코드, 라이브러리, 설정 파일 등)을 하나의 컨테이너(container) 라는 독립된 단위로 묶어, 어디서나 동일한 환경에서 실행할 수 있도록 해주는 가상화 기술이다. (빠른 배포, 이식성(Portability), 경량화된 가상환경 (VM보다 훨씬 가벼움))

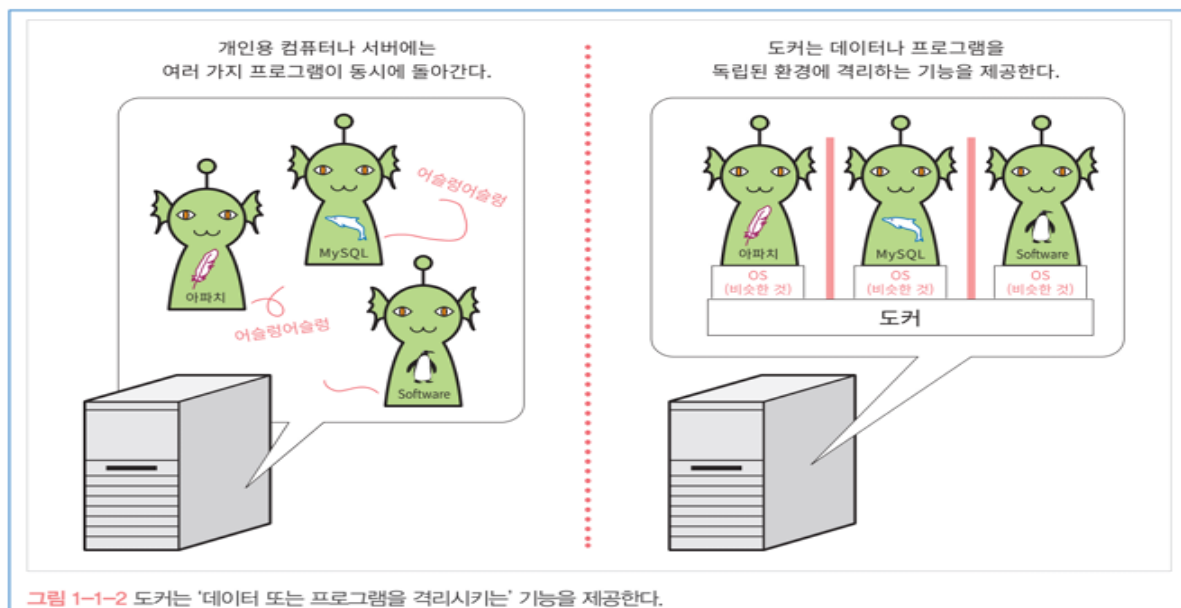
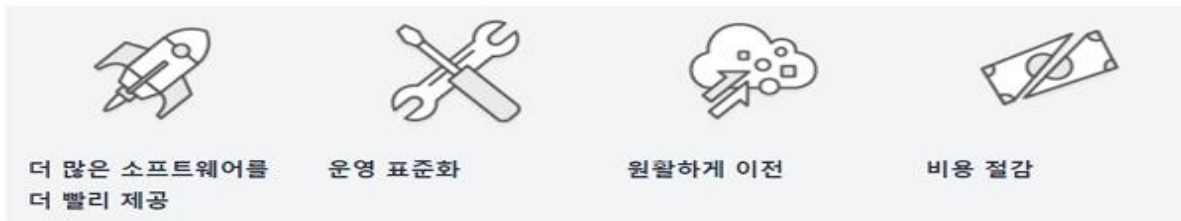


그림 1-1-2 도커는 '데이터 또는 프로그램을 격리시키는' 기능을 제공한다.

결론적으로 도커는



도커의 성질

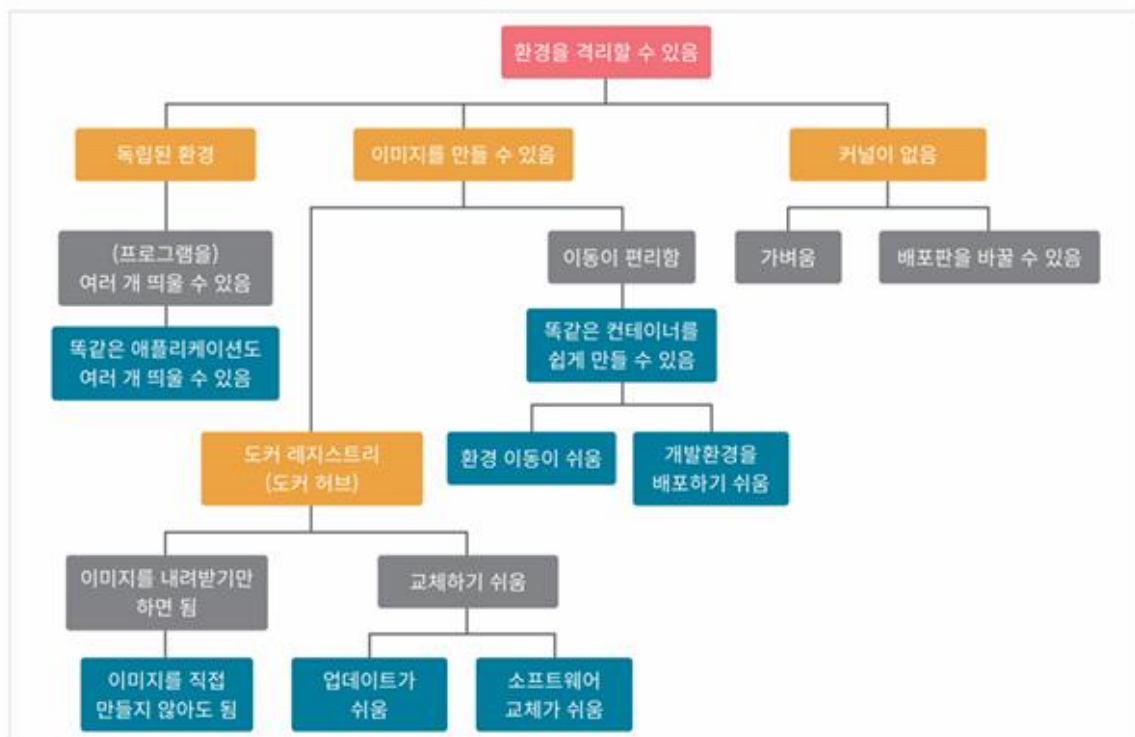


그림 2-4-1 도커의 성질

도커의장점 / 단점

장점

- 1대의 물리적인 서버(Host PC)에 여러 대의 서버를 구동할 수 있다.
- 다른 소프트웨어에 영향을 미치지 않으면서 컨테이너 교체나 수정이 쉬워 서버관리가 용이하다.
- 명령어 한줄로 서버 구축이 가능 하기에 서버 고수가 아니어도 다루기 수월하다.

단점

- 리눅스 소프트웨어만 지원한다.
- 호스트 서버에 문제발생시 모든 컨테이너에 영향을 미친다.
- 컨테이너를 하나만 사용하는 경우에는 비효율적이다.

도커의 주요 활용

팀원들에게 동일한 개발환경 제공

- 함께 일하는 프로젝트 팀원들에게 개발환경을 동일하게 제공
- 프로젝트별 컨테이너를 따로 사용할 수 있음

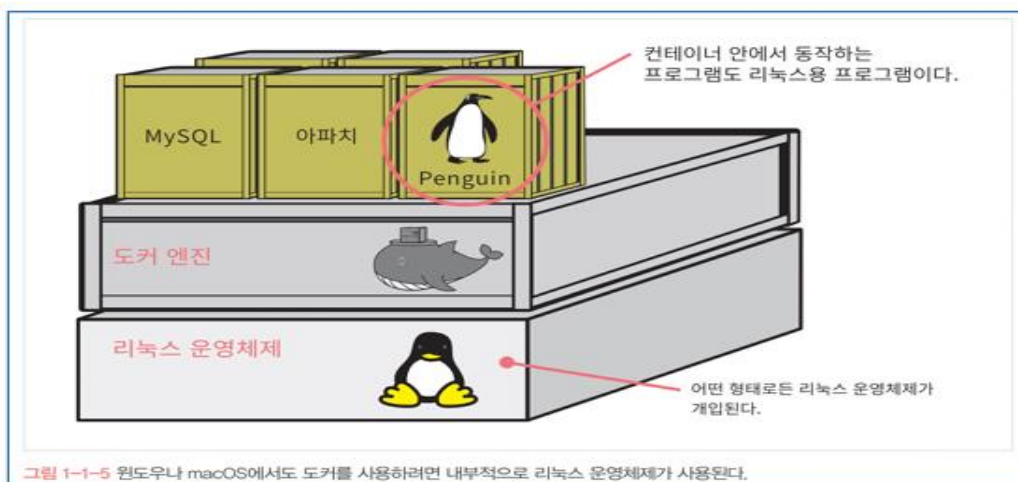
새로운 버전의 테스트

- 새로운 운영체제나 라이브러리 버전을 개발환경에서 먼저 테스트 후 운영환경에 컨테이너로 적용

동일한 서버가 여러 대 필요한 경우

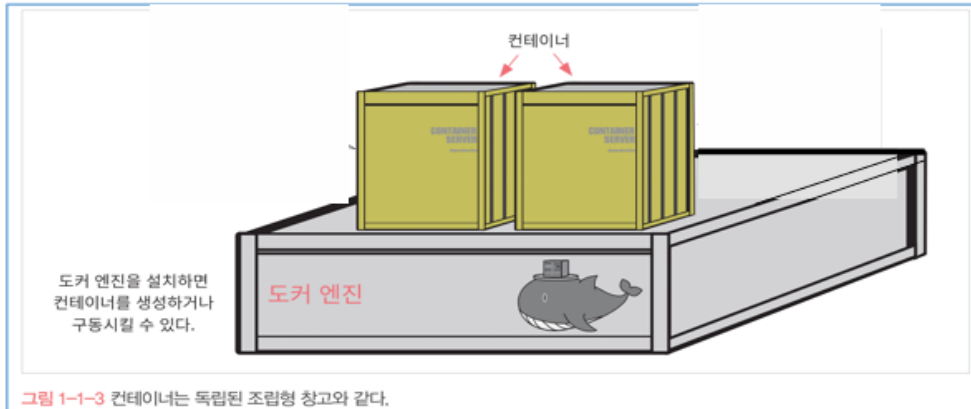
- 1대의 물리 서버에 여러 개의 컨테이너를 이용할 수 있으므로 비용 절약
- 명령어 한 줄로 원하는 만큼 서버 구동 가능 -> **배포시간 단축**
- **스케일링**에 유리하므로 웹 서버나 API 서버로 많이 활용

도커는 리눅스 운영체제가 필요하며, 도커는 리눅스 기반을 전제로 만들어졌다.

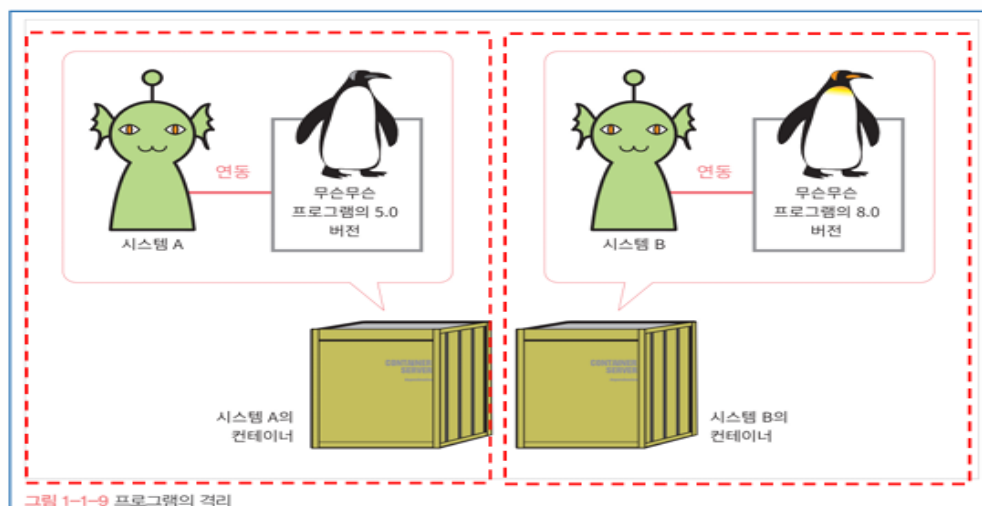


도커(Docker) - 컨테이너를 다루는 기능을 제공하는 소프트웨어

1)도커엔진이 필요하다.



2)컨테이너 생성을 위해 이미지 필요하다.



컨테이너 안의 프로그램은 다른 프로그램과 격리된 상태

완전히 독립된 환경이므로 동일한 버전 또는 다른 버전의 같은 프로그램 수행 가능

도커는 주로 서버용으로 많이 사용한다.

즉, 주로 정보를 제공하는 역할을 담당하는 서비스로 많이 사용한다.

서버란 '어떤 서비스를 제공하는 것'을 의미



그림 1-2-1 서버란 무엇인가?

서버의 대표적인 예

서버의 종류	설명
웹 서버	웹 사이트 기능을 제공하는 서버. HTML 파일 또는 이미지 파일, 프로그램을 배치하고, 클라이언트 컴퓨터의 브라우저에서 접근해오면 이들 파일을 제공한다. 대표적인 소프트웨어로 아파치, Nginx, IIS 등이 있다.
메일 서버	메일 송수신을 담당하는 SMTP 서버와 클라이언트에 메일을 전달하는 POP 서버로 나뉜다. 이들을 합쳐 메일 서버라고 부르는 경우가 많다. 메일을 내려받지 않고 서버에 둔 채로 읽는 IMAP4 서버도 있다. 대표적인 소프트웨어로 Sendmail, Postfix, Dovecot이 있다.
데이터베이스 서버	데이터를 저장하거나 검색하는 데이터베이스 기능을 제공하는 서버다. 대표적인 소프트웨어로 MySQL, PostgreSQL, MariaDB, SQL Server, Oracle Database가 있다.
파일 서버	파일을 저장하고 다른 사람과 공유하기 위한 서버다. 대표적인 소프트웨어로 Samba가 있다.
DNS 서버	IP 주소와 도메인을 연결해주는 DNS 기능을 제공하는 서버.
DHCP 서버	IP 주소를 자동으로 할당하는 기능을 제공하는 서버.
FTP 서버	FTP 프로토콜을 사용해 파일 송수신 기능을 제공하는 서버. 웹 서버와 함께 설치하는 경우가 많으며, 파일을 제공하는 데 사용한다.
프락시 서버	통신을 중계하는 역할을 맡는 서버의 통칭. 사내 로컬 네트워크 등 인터넷과 격리된 장소에서 인터넷 상의 서버에 접속할 때 사용한다. 프락시 서버를 경유하면 접속 대상 컴퓨터에 자신의 정체를 숨길 수 있으므로 정체를 드러내고 싶지 않을 때도 사용한다.
인증 서버	사용자 인증을 위한 서버. 윈도우 네트워크에 로그인하기 위한 Active Directory 서버 또는 무선 LAN이나 원격 접속에서 사용자 인증을 맡는 Radius 서버 등이 있다. 대표적인 소프트웨어로 OpenLDAP, Active Directory가 있다.

컨테이너 기술을 이용하면

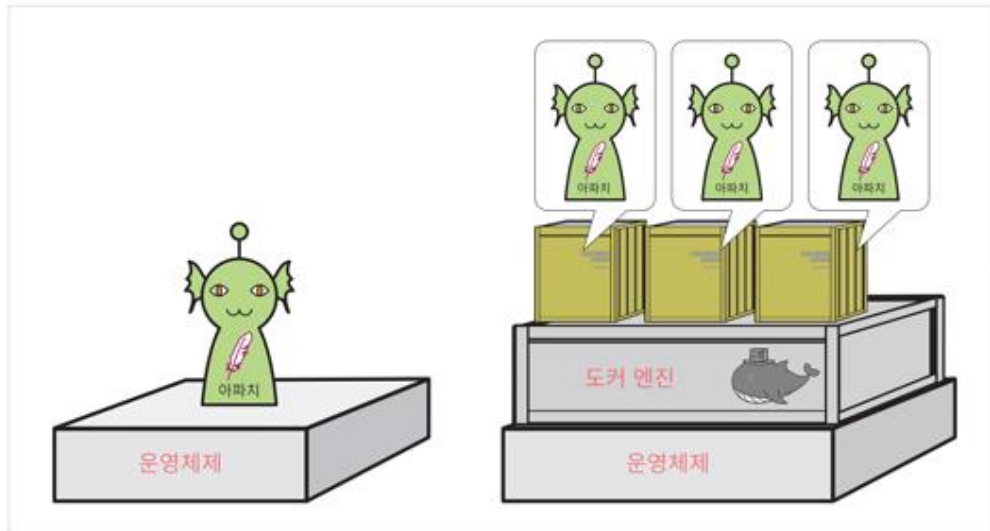


그림 1-2-8 컨테이너 기술을 활용하면 하나의 물리 서버에 여러 개의 웹 서버를 띄울 수 있다.

- 물리적인 1대의 컴퓨터에 웹서버, 메일서버, DB서버를 각 독립적 인 환경에서 수행
- 1대의 서버에서 여러 개의 웹서버 구동 가능
- 물리 서버의 수를 줄일 수 있으므로 비용 절감
- 개발 환경을 갖추거나 운영환경으로 쉽게 옮길 수 있다는 특성

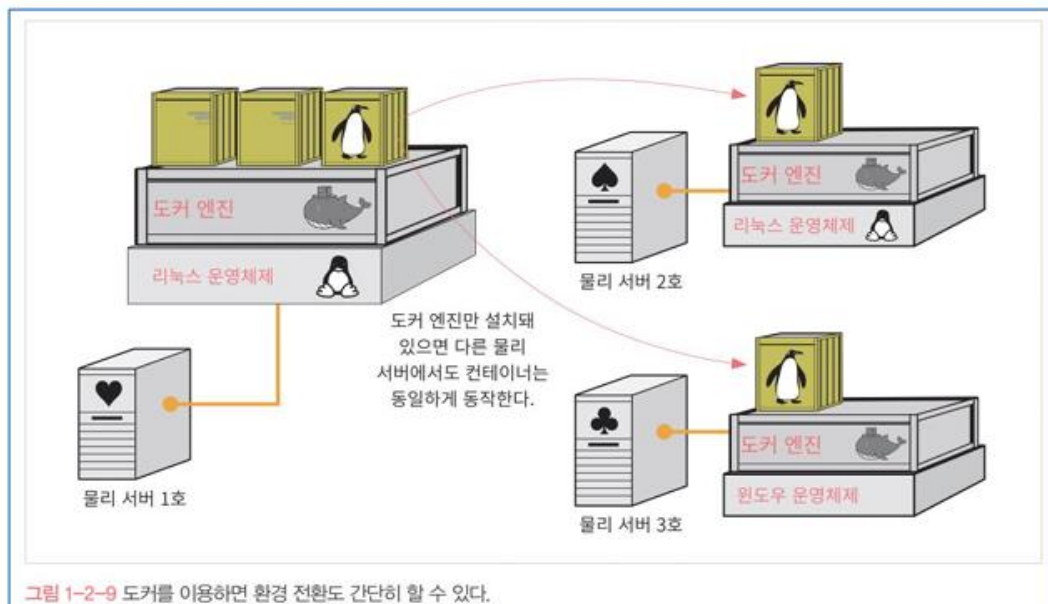


그림 1-2-9 도커를 이용하면 환경 전환도 간단히 할 수 있다.

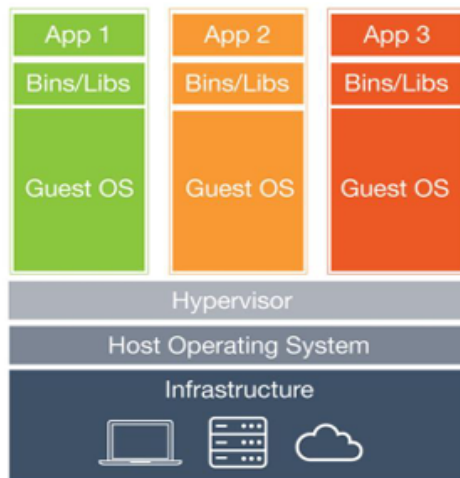
- 동일한 상태로 튜닝한 컨테이너를 팀원 전체에게 배포하여 동일한 개발 환경 사용이 용이
- 운영체제 무관, 도커만 설치되어 있으면 가능
- 물리적 환경의 차이나 서버 구성의 차이 등 운영서버와 개발서버 사이의 환경 차이로 인한 문제를 근본적으로 방지할 수 있음

도커가 나오기 전에 위와 같은 상황이 필요할 때 가상머신을 주로 이용 했다.

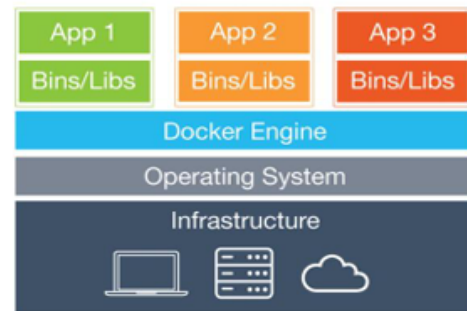
가상 머신

VS

도커 컨테이너 비교



OS와 하드웨어 드라이버를 host의 운영체제 즉 커널 단이 아닌 유저 단에 띄워서 가상화를 구현하는 방식이다. 가상화를 사용하는 주 목적은 Application을 사용하기 위함인데 매번 운영체제를 따로 설치해주는 것은 host 컴퓨터 입장에서는 부담스러운 일이다.



도커엔진을 이용하여 컨테이너라는 것을 띄운다. 컨테이너는 Application을 사용할 수 있는 가상 환경이고 별도의 운영체제가 필요하지 않다. 이를 통해 우리의 목적인 Application 사용도 가능하고 host환경에도 부담이 덜 간다.

하이퍼바이저는 호스트 컴퓨터에서 다수의 운영 체제를 동시에 실행하기 위한 논리적 플랫폼을 말한다. 가상화 머신 모니터 또는 가상화 머신 매니저라고도 부른다.

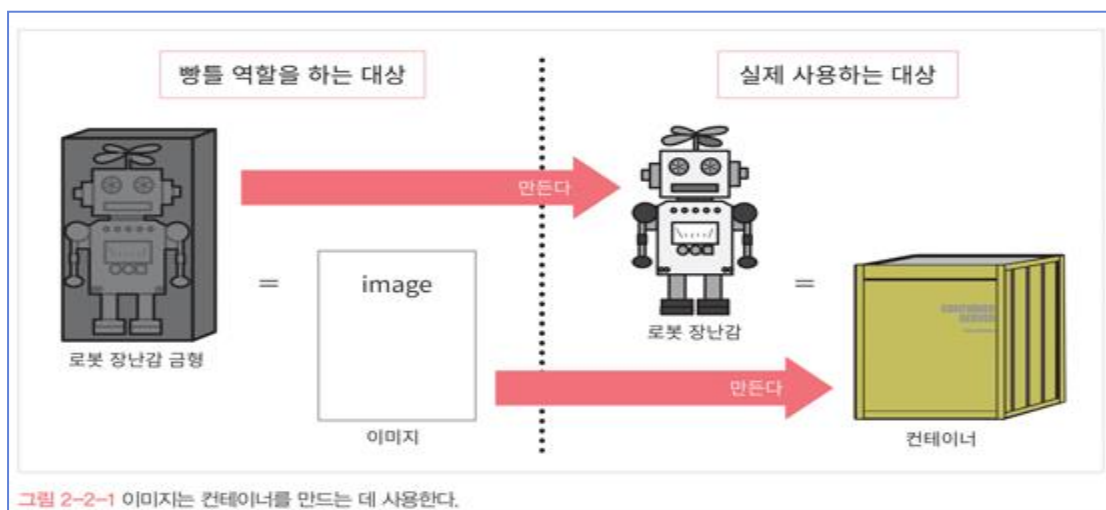


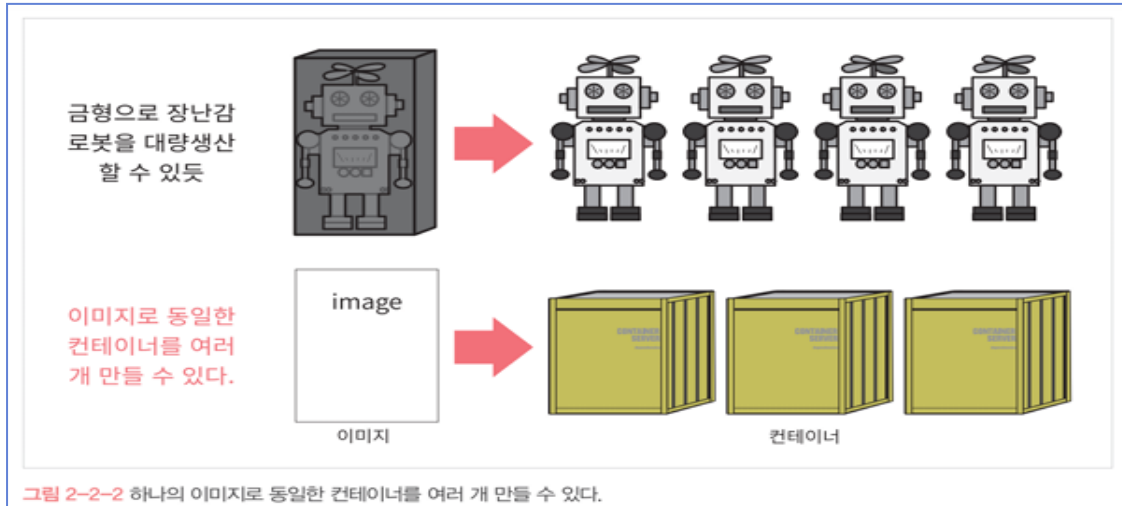


Docker 의 동작원리

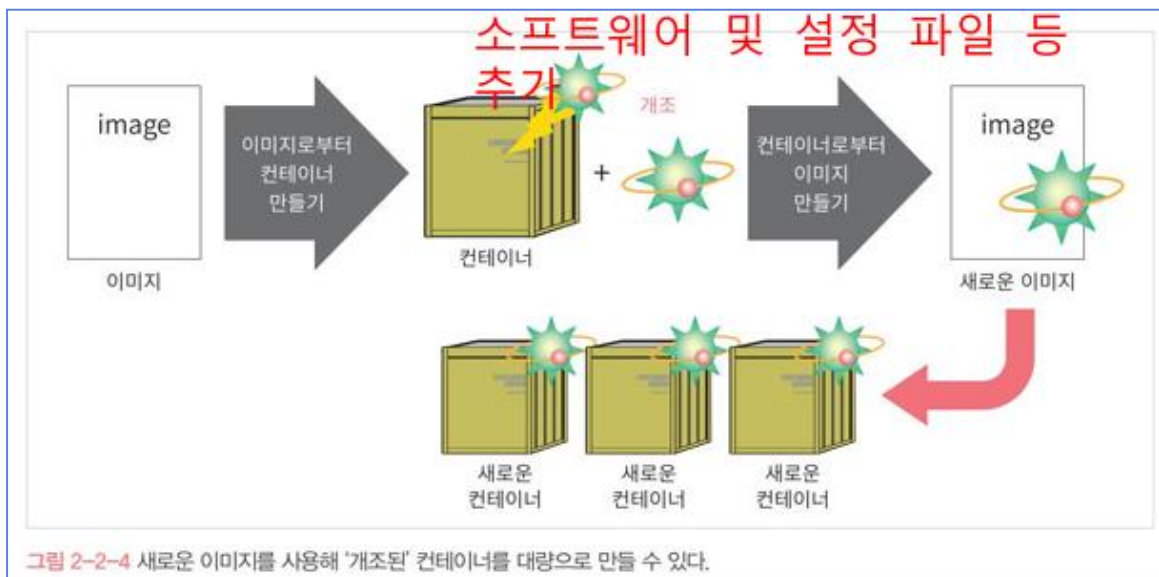
이미지란?

컨테이너의 설계도

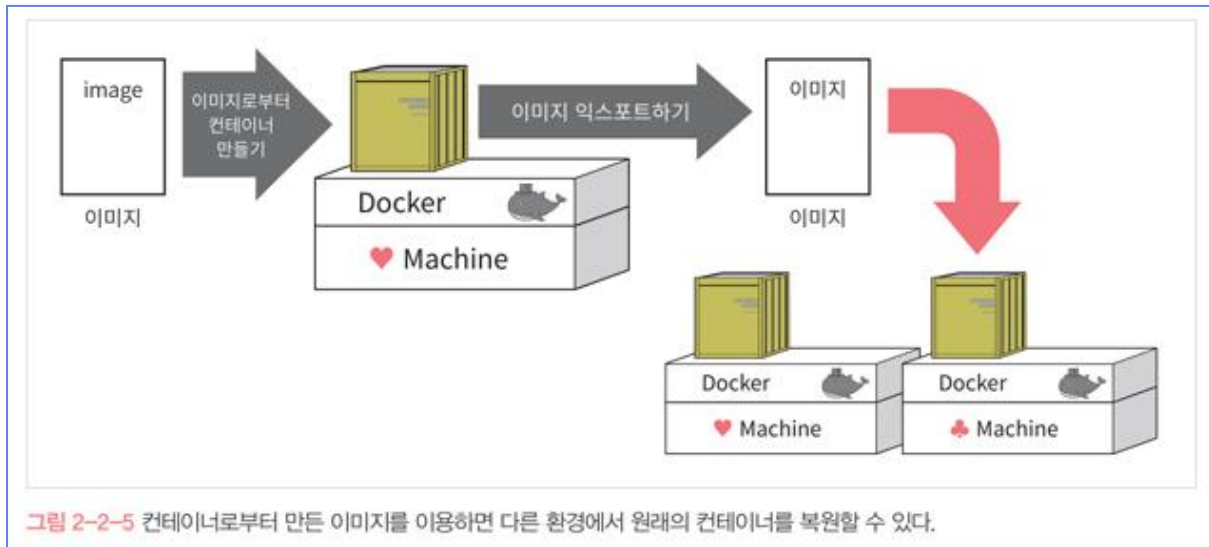




컨테이너로 이미지를 만들 수 있다.

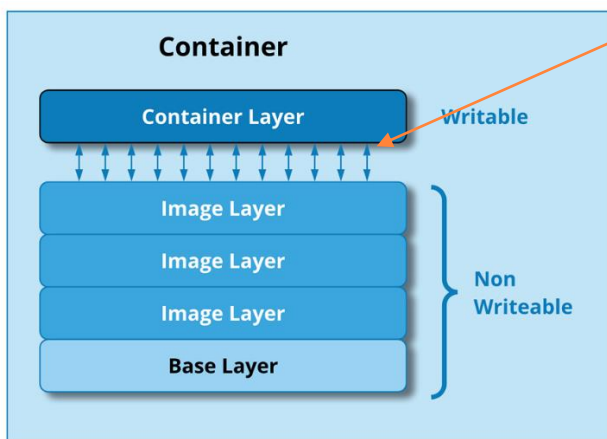


도커 엔진간 이동이 가능하다



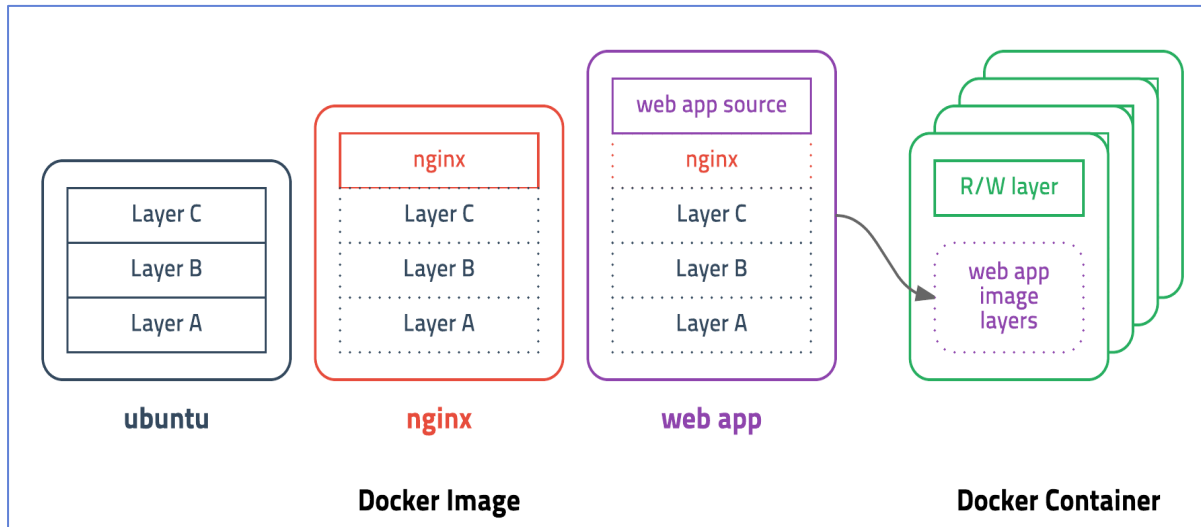
도커 이미지의 구조

Docker에서 이미지란 컨테이너를 구동시키는데 있어서 필요한 파일과 설정값 등을 저장한 파일 시스템이다. 이미지는 변하지 않고(Immutable) 상태를 가지지 않는다(Stateless). 이미지는 단지 컨테이너를 구동할 때만 사용되며, 실행 중에 변경된 내용은 컨테이너 레이어에 저장된다.

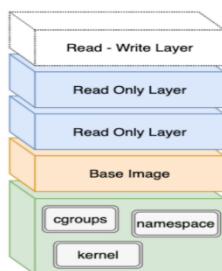


Docker 이미지의 핵심은 레이어 형태로 되어있다는 것이다. 유니온 파일 시스템을 이용해 여러 개의 레이어를 하나의 파일시스템으로 사용할 수 있게 해준다. 이미지는 여러 개의 읽기 전용 레이어로 구성되고 파일이 추가되거나 수정될 때 새로운 레이어가 생성된다.

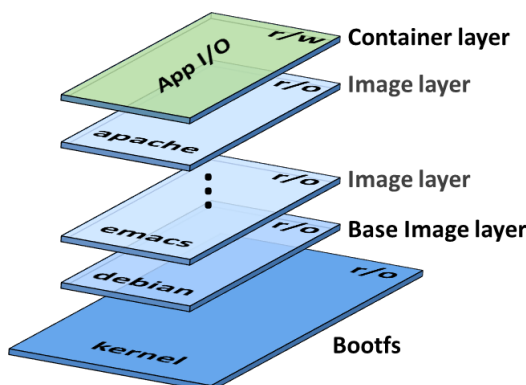
여러 이미지를 가지고 있다고 하여도 중복된 부분은 하나만 가지고 있어서 공간이 효율적으로 관리되고 확장성이 좋다. 이런 확장성을 바탕으로 Docker Hub에는 다양한 이미지들이 존재하고 80억이 넘는 다운로드가 이루어지고 있다.



ubuntu 이미지가 기존에 존재하는데 nginx 이미지를 다운 받을 경우 nginx 레이어만 다운받게 된다. Docker 이미지는 위 그림처럼 여러 레이어로 구성되며, 각 레이어는 이전 레이어의 변경 사항을 가지고 있다. 여러 개의 레이어에는 읽기 전용인 read only 레이어와, 새로 변경되거나 추가된 내용을 담은 새로운 레이어로 구성된다.



이미지를 실행하여 도커 컨테이너를 생성할 때도 레이어 방식을 사용한다. 컨테이너 생성 시, 기존의 읽기 전용 이미지 레이어 위에 읽기/쓰기 전용 레이어(R/W layer)를 추가한다.



```
C:\Windows\system32\cmd.exe
C:\deployeducation>docker run -p 80:80 nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
8a1e25ce7c4f: Already exists
e78b137be355: Pull complete
39fc875bd2b2: Pull complete
035788421403: Pull complete
87c3fb37cbf2: Pull complete
c5cdd1ce752d: Pull complete
33952c599532: Pull complete
Digest: sha256:6db391d1c0cfb30588ba0bf72ea9994
```

이미지의 레이어 구조는 재활용에 유리하며 효율적인 데이터 저장과 전송을 지원한다. 이미지 레이어는 이전 레이어의 변경 사항을 저장한다. 컨테이너 실행 시 읽기, 쓰기가 가능한 새로운 레이어가 추가된다.(컨테이너 레이어)

1. 이미지 레이어란?

- Docker 이미지는 여러 층(layer)으로 쌓여 만들어진다.
- 각 레이어는 **파일 시스템의 스냅샷** 같은 것이라고 생각하면된다.
- 예를 들어 Dockerfile에서 명령어 하나(FROM, RUN, COPY, ...)를 실행할 때마다 새로운 레이어가 만들어짐.

2. 중복된 레이어는 한 번만 저장된다

Docker는 같은 레이어를 여러 이미지에서 공유할 수 있다.

즉, 서로 다른 이미지를 다운받아도 **겹치는 레이어는 한 번만 저장된다.**

예시 상황

1. 내가 python:3.10 이미지를 받았다.
 - 내부 레이어:
 - Ubuntu 20.04 기반 OS 레이어
 - Python 3.10 설치 레이어
2. 또 python:3.11 이미지를 받는다.
 - 내부 레이어:
 - Ubuntu 20.04 기반 OS 레이어 (**이미 python:3.10에도 있음**)
 - Python 3.11 설치 레이어

여기서 **Ubuntu 20.04 레이어는 똑같으니까 한 번만 저장됨**

그래서 디스크 용량이 절약된다.

장점

- **저장 공간 절약:** 같은 기반 OS를 쓰는 여러 이미지가 있어도, OS 부분은 한 번만 저장.
- **속도 향상:** 이미 있는 레이어는 다시 다운로드할 필요가 없음 → docker pull이 빠름.
- **빌드 효율:** 캐시된 레이어를 재활용할 수 있어 재빌드 속도도 빨라짐.

이미지 다운로드 과정의 원리**이미지 검색 및 선택**

- "docker image pull" 명령을 사용하여 도커 허브(Docker Hub)와 같은 이미지 레지스트리에서 이미지를 검색하고 선택함.

이미지 레이어 다운로드

- 도커 이미지는 여러 개의 레이어로 구성됨.
- 각 레이어는 파일 시스템의 변화를 표현하며, 이전 레이어에서 변경된 내용만 저장됨.
- 이미 다운로드한 이미지의 레이어는 캐시에 저장되어 중복 다운로드를 방지함.

레이어 병합

- 다운로드한 레이어들은 로컬 머신에서 하나의 이미지로 병합됨.
- 각 레이어는 변경이 불가능하므로, 이미지 자체는 변경 불가능한 단위로 관리됨.

이미지 생성 및 태깅

- 다운로드한 이미지를 기반으로 새로운 이미지를 생성할 수 있음.
- "docker image tag" 명령을 사용하여 이미지에 태그를 부여하여 버전을 관리할 수 있음.

Docker Hub와 Docker Image

Docker Hub란?

Docker Hub는도커 이미지를 저장하고 공유할 수 있는 "클라우드 저장소" 이다

Docker Image 파일(컨테이너를 만들 때 필요한 설계도)을 인터넷에 올리고, 다운로드할 수 있게 해주는Docker 공식 저장소 서비스이다.

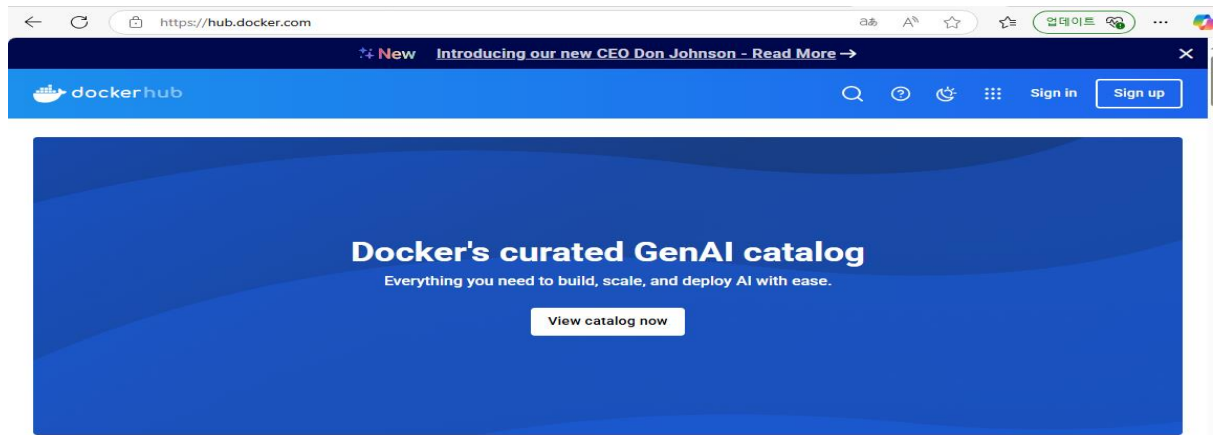
마치 GitHub가 소스코드를 저장하고 공유하는 곳이라면,Docker Hub는 Docker 이미지를 저장하고 공유하는 곳이다.

Docker Hub에서 할 수 있는 일

기능	설명
이미지 검색	다른 사람이 만든 이미지를 검색해서 다운받을 수 있음 (ex. <code>mysql</code> , <code>nginx</code> , <code>ubuntu</code>)
이미지 업로드(Push)	내가 만든 이미지를 Docker Hub에 업로드해서 공유할 수 있음
이미지 다운로드(Pull)	필요한 이미지를 내 컴퓨터에 다운로드할 수 있음
이미지 관리	내 계정에 저장된 이미지 목록을 관리할 수 있음
공개/비공개 설정	이미지를 모두에게 공개하거나 나만 볼 수 있게 비공개 설정 가능

도커 허브(<https://hub.docker.com>)

회원가입하기



안전한 컨테이너 이미지 고르는 방법

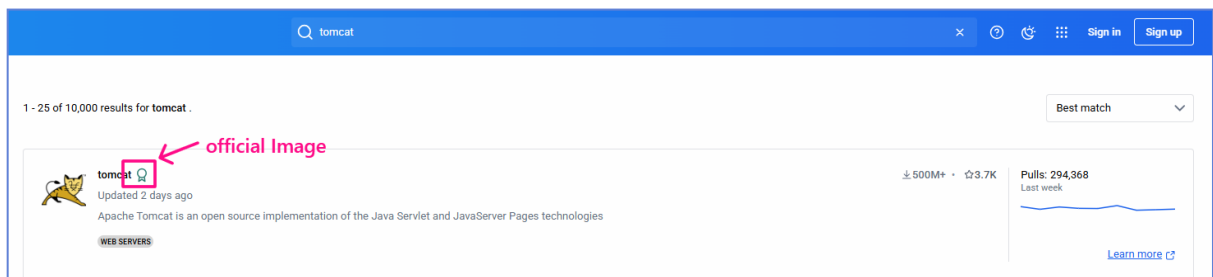
: 공식 이미지를 사용한다(Official Image)

도커에서 공식적으로 관리하거나 공식 파트너와 함께 유지 보수하는 것으로 신뢰 할 수 있도록 제공하는 공식 인증된 컨테이너 이미지

Ex) 오피셜 : mysql:8.4 (오피셜은 도커허브계정명 없다.)

비공식 : someuser/mysql:8.4

: 커스텀 이미지를 직접 만들어 사용한다



다양한 형태로 조합이 가능한 컨테이너

도커 사용 원칙 '한 컨테이너에 한 프로그램'을 사용하는 것을 권장한다.

보안 및 유지 관리 측면에서 유리하다.



그림 2-2-11 한 컨테이너에 하나의 프로그램 또는 여러 개의 프로그램을 넣을 수도 있다.

도커 컨테이너의 생애 주기

:도커 컨테이너는 **일회용품**으로 생각하자.

: 업데이트시 새로운 이미지로부터 새로운 컨테이너를 만들어 배포하는 방식을 이용한다.

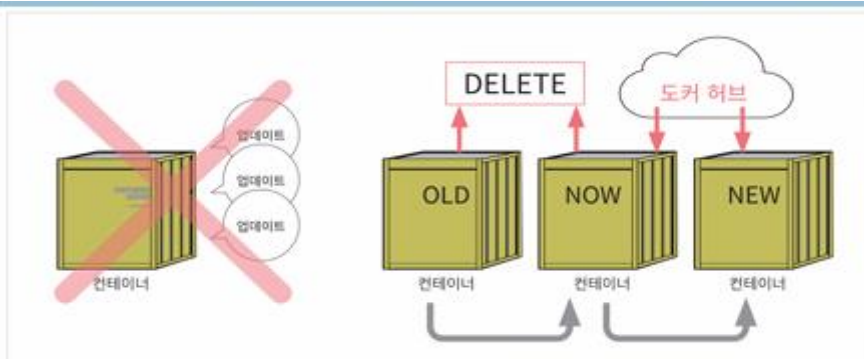


그림 2-3-1 컨테이너는 '쓰고 버리는' 일회용품에 가깝다.

What is Container LifeCycle?

컨테이너를 만들고 ⇒ 실행하고 ⇒
종료하고 ⇒ 폐기하는 과정
= 컨테이너의 생애주기

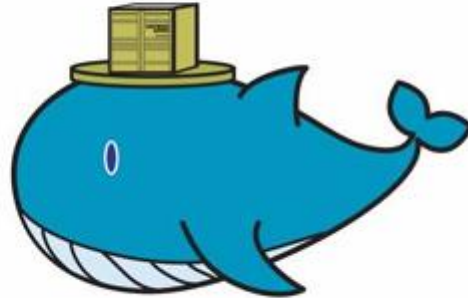


그림 2-3-2 컨테이너의 생애주기

도커 컨테이너가 일회용품이라면

컨테이너 삭제시 그 안에 있던 파일들은 어떻게 관리해야 하나?

일반적으로 물리적인 서버의 디스크를 **마운트(볼륨마운트 or 바인드마운트)**하여 저장한다.

마운트 하면 컨테이너를 삭제해도 데이터를 외부에 안전하게 보관할 수 있고 도커 엔진에 장애가 발생해도 데이터는 그대로 보존된다.

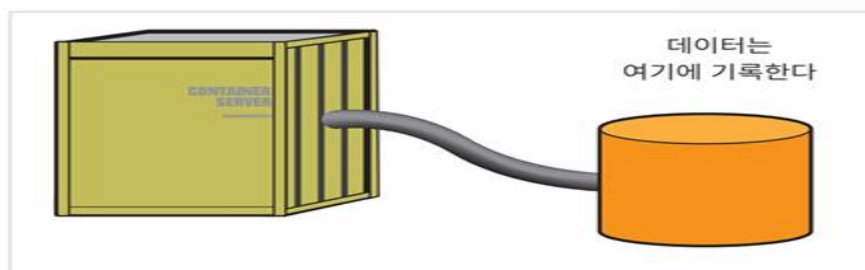
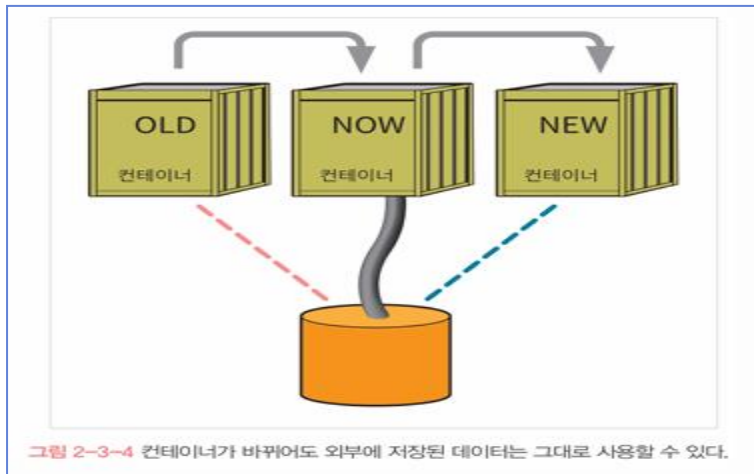
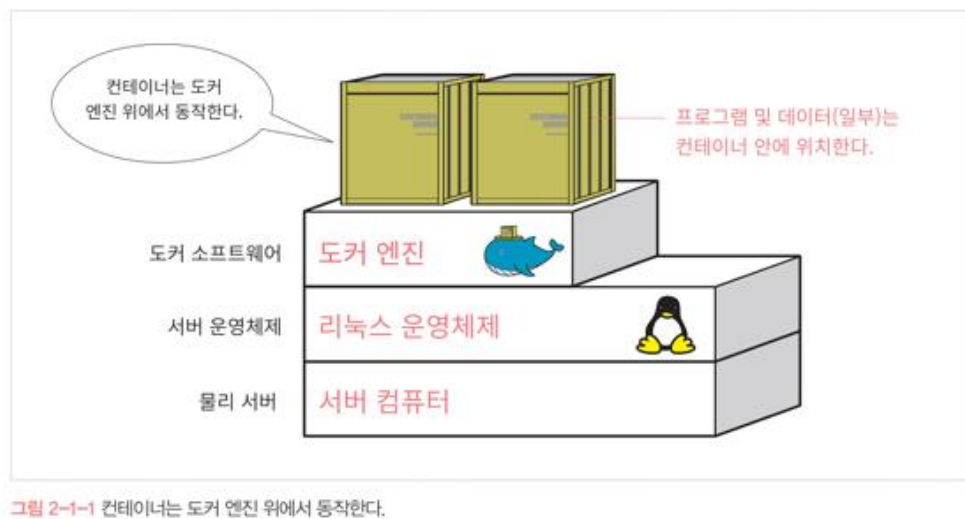


그림 2-3-3 물리적 컴퓨터의 일부를 마운트하고 여기에 데이터를 저장한다.

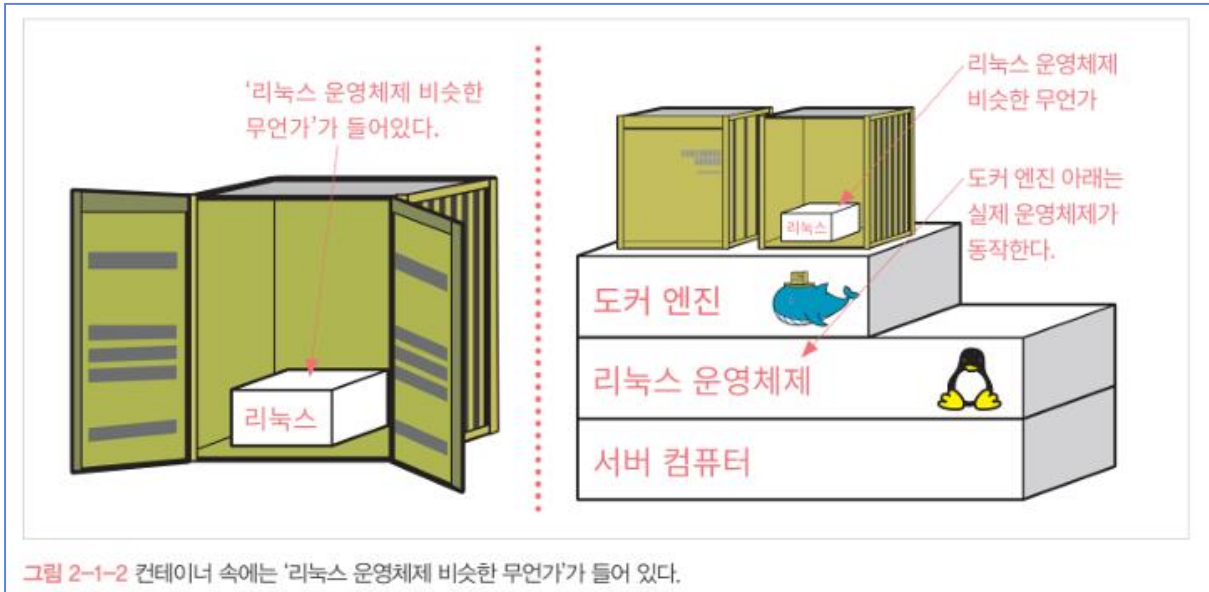
컨테이너가 바뀌어도 데이터는 계속 이용할 수 있다



도커의 구조



운영체제 위에 도커 엔진이 동작하고 그 위에서 컨테이너가 동작하며 프로그램과 데이터는 컨테이너 안에 위치한다.



모든 컨테이너 안에는 리눅스 운영체제와 유사한 환경 제공한다.

운영체제의 구조

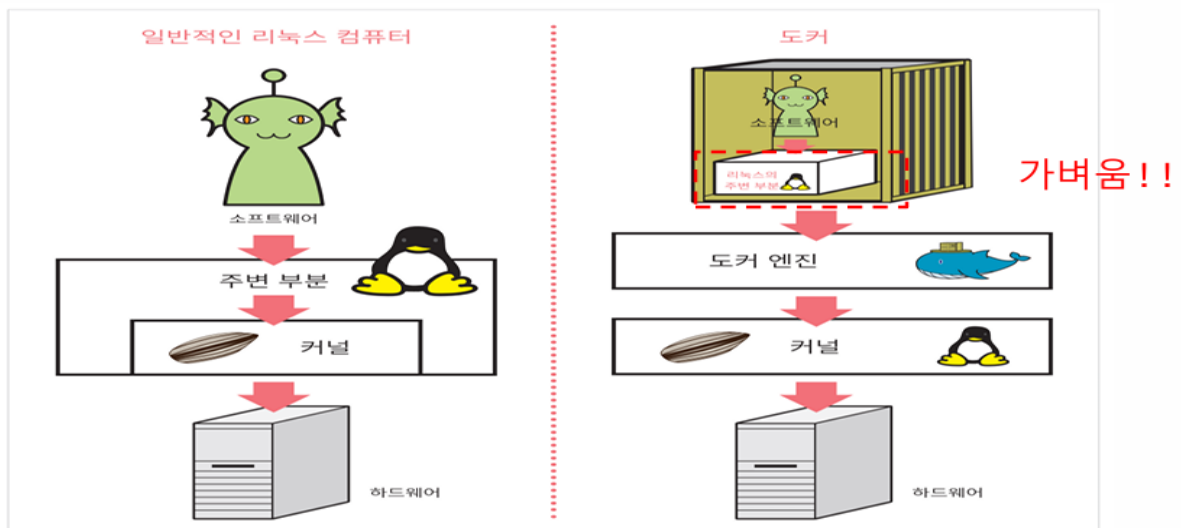


그림 2-1-4 컨테이너는 외부와 완전히 분리돼 있으므로 컨테이너 속에 운영체제의 주변 부분이 따로 들어 있다.

도커는 기본적으로 리눅스 운영체제에서만 동작하기 때문에 컨테이너에서 실행할 소프트웨어 역시 리눅스용 소프트웨어이다. 즉, 도커는 리눅스 컴퓨터에 독립된 격리 환경을 만드는 것이다.

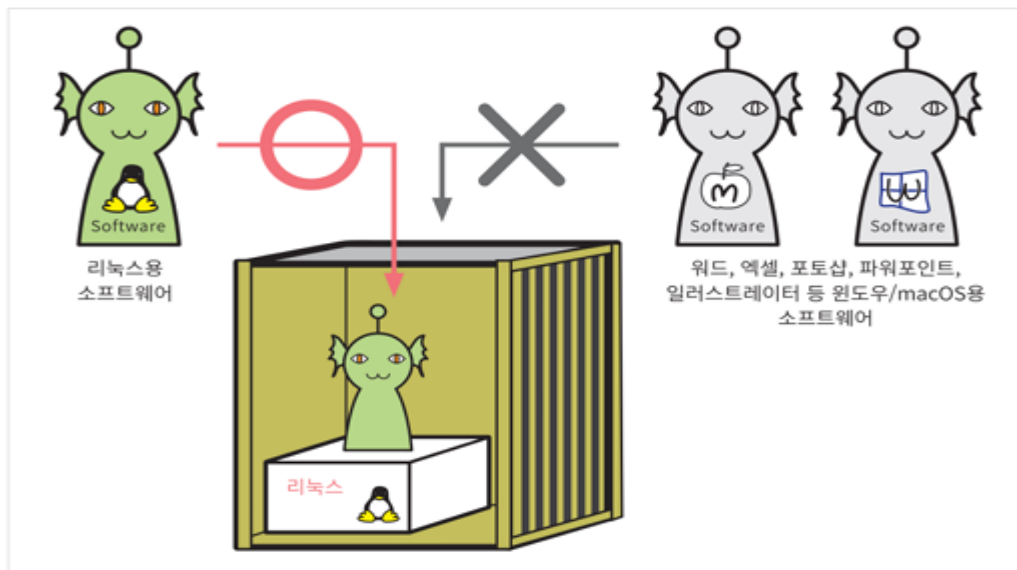


그림 2-1-5 컨테이너에 들어가는 프로그램은 리눅스용이어야 한다.

윈도우나 macOS위에서 도커를 수행하려면 리눅스 운영체제를 갖추어야 한다.

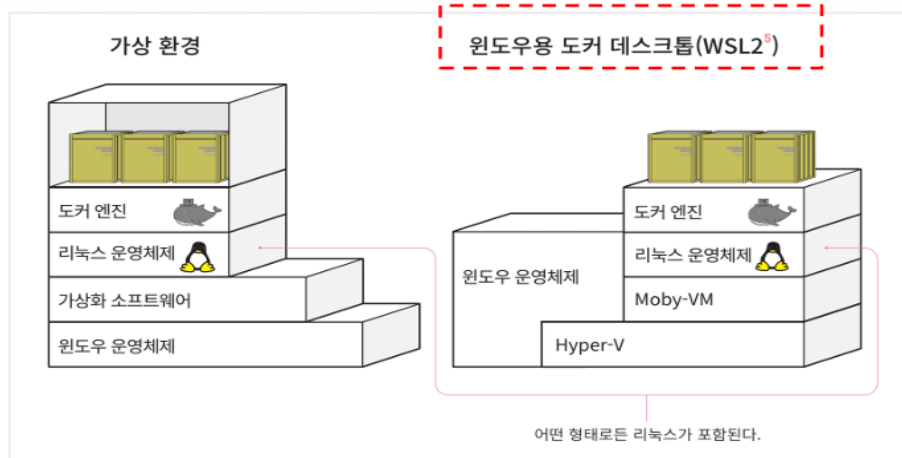


그림 2-1-6 도커는 기본적으로 리눅스용이다.

윈도우 환경에서 WSL2(Windows Subsystem for Linux 2)와 Docker Desktop을 이용해서 Docker를 할 수 있다.

구분	Docker Desktop	WSL2
목적	Docker를 쉽게 설치하고 관리할 수 있는 통합 프로그램	Windows에서 리눅스를 실행할 수 있도록 만들어 준 운영체제 기능
하는 일	Docker Engine 설치/관리, 컨테이너 실행, GUI 제공 등 전체 Docker 환경 제공	리눅스 커널을 Windows 안에 띄워서 리눅스 명령어, 리눅스 프로그램 사용 가능하게 함
종류	응용 프로그램(소프트웨어)	운영체제 기능 (Windows 시스템 기능)
Docker와의 관계	Docker Desktop이 Docker Engine을 WSL2 안에 설치해서 사용하는 구조	WSL2는 Docker가 동작할 리눅스 환경을 제공할 (WSL2만으로는 Docker를 직접 제공하지 않음)
필수 여부	Windows에서 Docker를 쉽게 쓰려면 필수 (특히 GUI까지 쓰고 싶으면)	Docker Desktop 설치 시 자동 설치 요구 (또는 직접 설치 가능)

Docker Desktop

내부적으로 WSL2를 사용해서 리눅스 환경을 만들고 그 위에 Docker Engine을 설치해서 우리에게 Docker를 쓸 수 있게 해준다.

즉, Windows에서 Docker를 쓰려면 WSL2가 밑에 깔려 있어야 하고, Docker Desktop은 그 위에서 Docker를 쉽게 관리해 주는 도구다.